

HURDLE 3

AI Carbon Forecasting Model

Train a time-series model to predict next-quarter carbon emissions.

 35 Minutes

Objective

Train a machine learning model on historical daily CO2e data to forecast carbon emissions for the next 30 days. Evaluate the model and save the trained artefact so it can be loaded by the API in Hurdle 4.

Background

Reactive sustainability is not enough — Xebia's teams need to anticipate carbon spikes before they happen. A time-series forecasting model turns historical usage patterns into forward-looking predictions, enabling proactive infrastructure decisions. This is the AI core of the GreenOps Dashboard.

Tasks

Task A — Feature Engineering

1. Load `cloud_usage_enriched.csv`. Aggregate to daily total CO2e: `daily = df.groupby('date')['co2e_kg'].sum().reset_index()`
2. Create lag features: `daily['lag_7'] = daily['co2e_kg'].shift(7)`, `daily['lag_14'] = daily['co2e_kg'].shift(14)`
3. Create a rolling mean: `daily['rolling_7'] = daily['co2e_kg'].rolling(7).mean()`
4. Create a day-of-week feature: `daily['dow'] = daily['date'].dt.dayofweek`
5. Drop NaN rows introduced by shifting.

Task B — Train/Test Split & Model Training

6. Use the last 30 days as the test set, everything else as training.
7. Define features `X = ['lag_7', 'lag_14', 'rolling_7', 'dow']` and target `y = 'co2e_kg'`.
8. Train a Linear Regression model:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
```

9. (Stretch) Also try `RandomForestRegressor` and compare RMSE.

Task C — Evaluate the Model

10. Predict on the test set: `y_pred = model.predict(X_test)`
11. Calculate RMSE: from `sklearn.metrics` import `mean_squared_error`; `rmse = mean_squared_error(y_test, y_pred, squared=False)`
12. Print RMSE and interpret: is the error < 10% of mean daily CO2e? If not, what might help?
13. Plot actual vs predicted CO2e for the 30-day test period and save as `model/forecast_plot.png`.

Task D — Save & Commit Model Artefact

14. Save the trained model: `import joblib; joblib.dump(model, 'model/co2e_model.pkl')`
15. Commit model artefact, feature engineering script, and evaluation plot to GitHub.
16. Message: 'feat: CO2e forecasting model with lag features'



Concept Check

- What is RMSE and what does a lower value indicate?
- Why do we create lag features for time-series prediction instead of using the date directly?
- What are the risks of using Linear Regression for this task? What assumptions does it make?



Self-Evaluation Checklist

☐	Daily CO2e aggregation created; lag and rolling features engineered
☐	Train/test split applied (last 30 days = test)
☐	Linear Regression (or better) model trained successfully
☐	RMSE calculated and interpreted in comments/README
☐	Actual vs predicted plot saved to <code>model/forecast_plot.png</code>
☐	<code>co2e_model.pkl</code> committed to GitHub
☐	Concept check answered

» Proceed to Hurdle 4 once checklist is complete.